RUDN university

Reinforcement Learning with Artificial neural network

Lecture 3

Prepared by

Lect. Safa Abdulwahab Hameed

Department of Mechanics and control processes

Academy of Engineering

Outline

- **Elements of Reinforcement Learning**
- **Example: Tic-Tac-Toe**
- **Exercises**



Elements of Reinforcement Learning

Beyond the agent and the environment, one can identify four main sub elements of a reinforcement learning system:

a policy, a reward signal, a value function, and, optionally, a model of the environment.

A policy defines the learning agent's way of behaving at a given time.

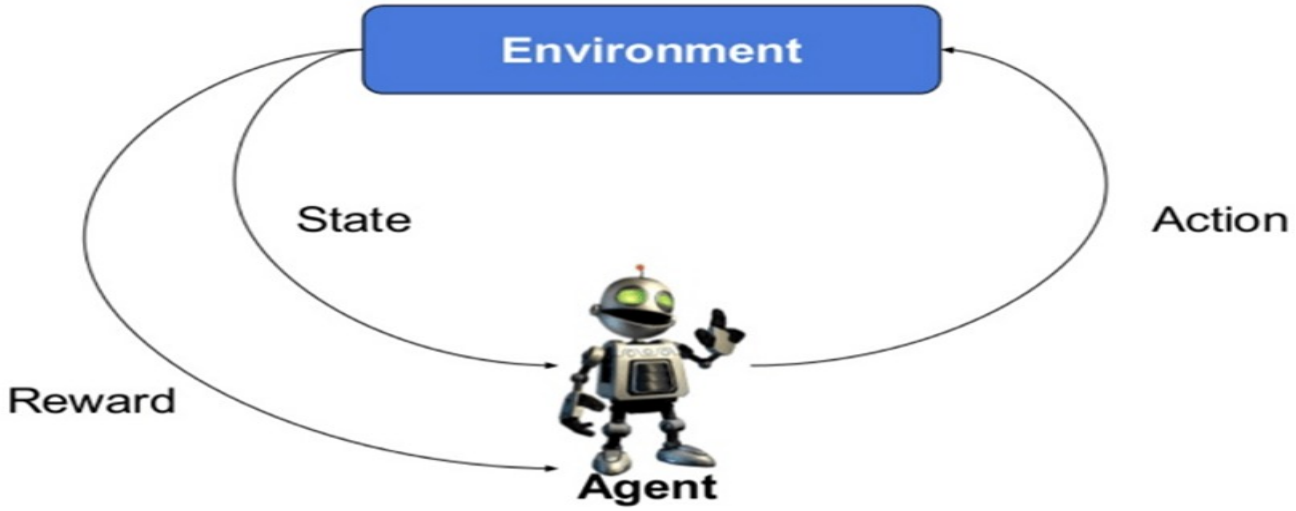
Roughly speaking, a policy is a mapping from perceived states of the environment to actions to be taken when in those states.



In some cases the policy may be a simple function or lookup table, whereas in others it may involve extensive computation such as a search process.

The policy is the core of a reinforcement learning agent.

In general, policies may be stochastic, specifying probabilities for each action.



A reward signal defines the goal of a reinforcement learning problem.

On each time step, the environment sends to the reinforcement learning agent a single number called the reward.

The agent's role objective is to maximize the total reward it receives over the long run.

The reward signal thus defines what are the good and bad events for the agent.



In a biological system, we might think of rewards as analogous to the experiences of pleasure or pain.

They are the immediate and defining features of the problem faced by the agent.

The reward signal is the primary basis for altering the policy; if an action selected by the policy is followed by low reward, then the policy may be changed to select some other action in that situation in the future.



In general, reward signals may be stochastic functions of the state of the environment and the actions taken.

Whereas the reward signal indicates what is good in an immediate sense, a value function specifies what is good in the long run.

Roughly speaking, the value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state.

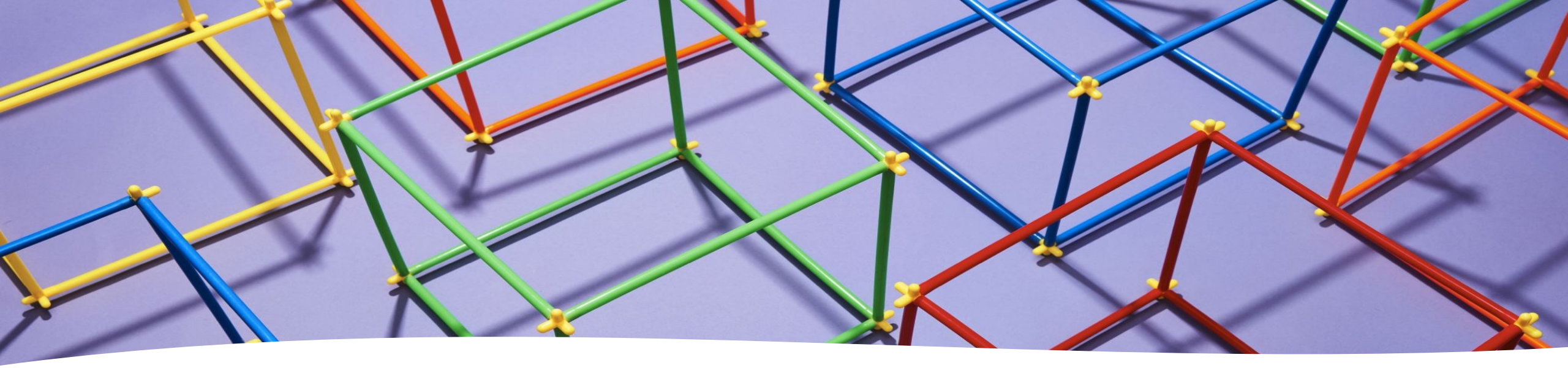


Rewards are in a sense primary, whereas values, as predictions of rewards, are secondary.

Without rewards there could be no values, and the only purpose of estimating values is to achieve more reward.

Nevertheless, it is values with which we are most concerned when making and evaluating decisions.

Action choices are made based on value judgments.



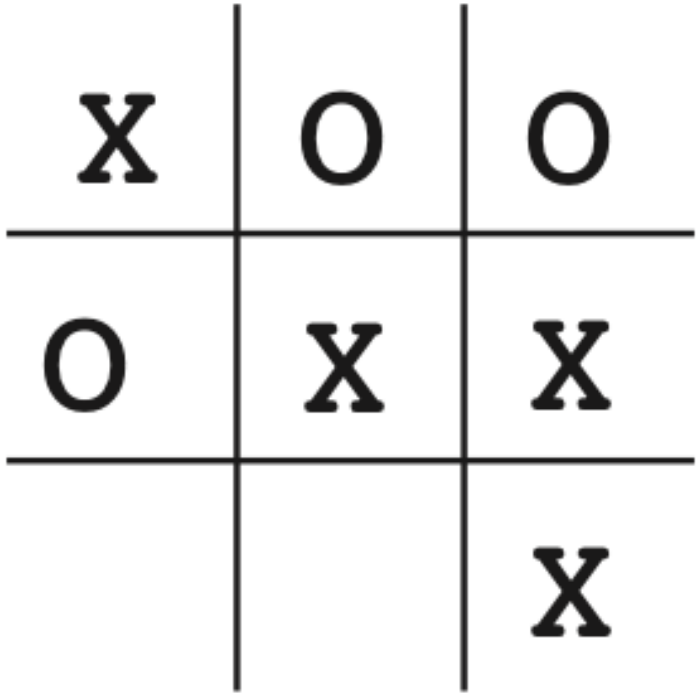
The fourth and final element of some reinforcement learning systems is a model of the environment.

This is something that mimics the behavior of the environment, or more generally, that allows inferences to be made about how the environment will behave.

For example, given a state and action, the model might predict the resultant next state and next reward.

Models are used for planning, by which we mean any way of deciding on a course of action by considering possible future situations before they are actually experienced.

Methods for solving reinforcement learning problems that use models and planning are called model-based methods, as opposed to simpler model-free methods that are explicitly trial-and-error learners—viewed as almost the opposite of planning.



Example: Tic-Tac-Toe

To illustrate the general idea of reinforcement learning, consider the familiar child's game of tic-tac-toe.

Two players take turns playing on a three-by-three board.

One player plays Xs and the other Os until one player wins by placing three marks in a row, horizontally, vertically, or diagonally, as the X player has in the game shown.

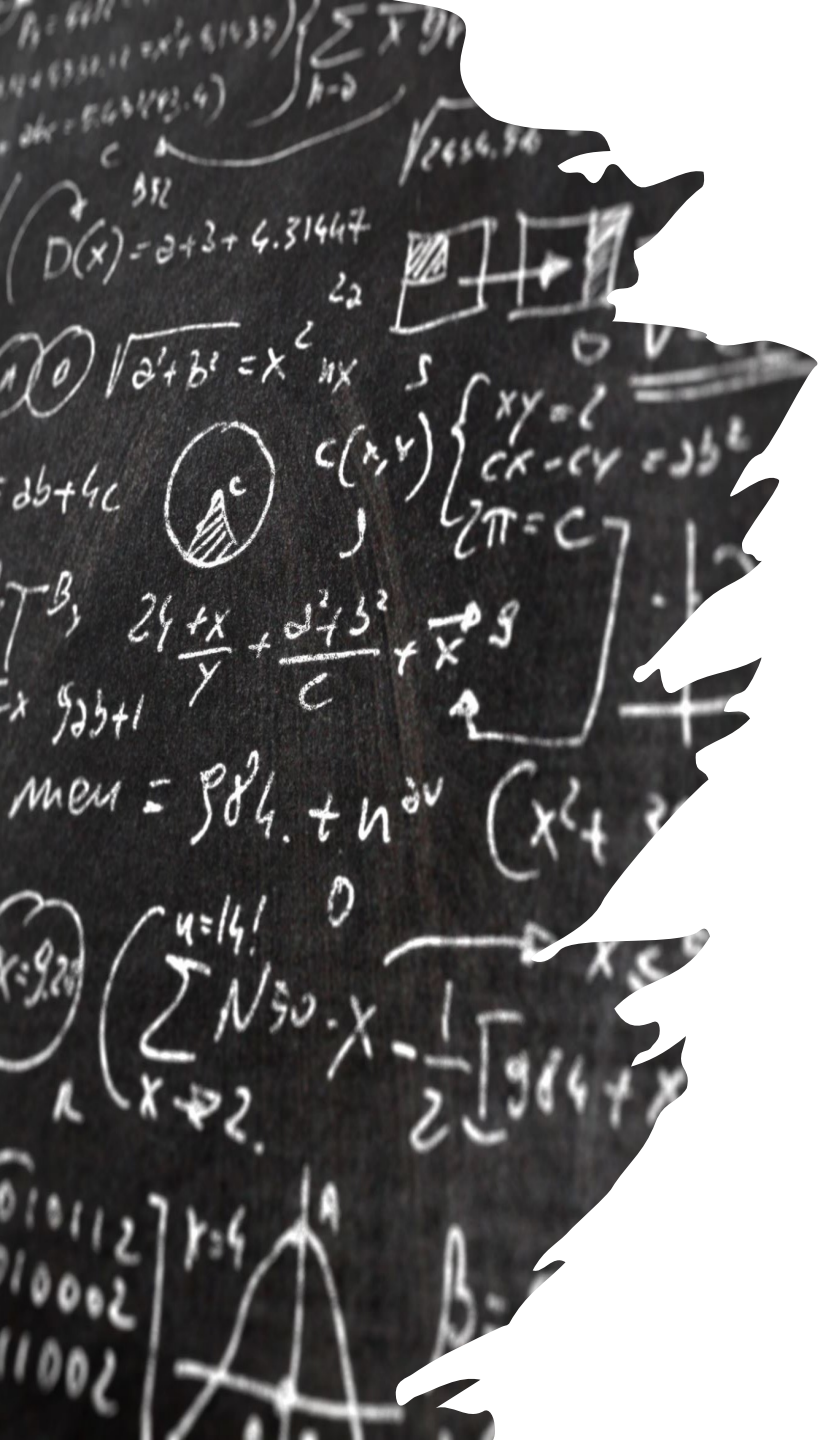


If the board fills up with neither player getting three in a row, then the game is a draw.

Because a skilled player can play so as never to lose, let us assume that we are playing against an imperfect player, one whose play is sometimes incorrect and allows us to win.

For the moment, in fact, let us consider draws and losses to be equally bad for us.

How might we construct a player that will find the imperfections in its opponent's play and learn to maximize its chances of winning?



Classical optimization methods for sequential decision problems, such as dynamic programming, can compute an optimal solution for any opponent, but require as input a complete specification of that opponent, including the probabilities with which the opponent makes each move in each board state.

Let us assume that this information is not available a priori for this problem, as it is not for the vast majority of problems of practical interest.

On the other hand, such information can be estimated from experience, in this case by playing many games against the opponent.

About the best one can do on this problem is first to learn a model of the opponent's behavior, up to some level of confidence, and then apply dynamic programming to compute an optimal solution given the approximate opponent model.

An evolutionary method applied to this problem would directly search the space of possible policies for one with a high probability of winning against the opponent.

Here, a policy is a rule that tells the player what move to make for every state of the game—every possible configuration of Xs and Os on the three-by-three board.

For each policy considered, an estimate of its winning probability would be obtained by playing some number of games against the opponent.

This evaluation would then direct which policy or policies were considered next.

A typical evolutionary method would hill-climb in policy space, successively generating and evaluating policies in an attempt to obtain incremental improvements.

Or, perhaps, a genetic-style algorithm could be used that would maintain and evaluate a population of policies. Literally hundreds of different optimization methods could be applied.

Here is how the tic-tac-toe problem would be approached with a method making use of a value function.

First we would set up a table of numbers, one for each possible state of the game.

Each number will be the latest estimate of the probability of our winning from that state.

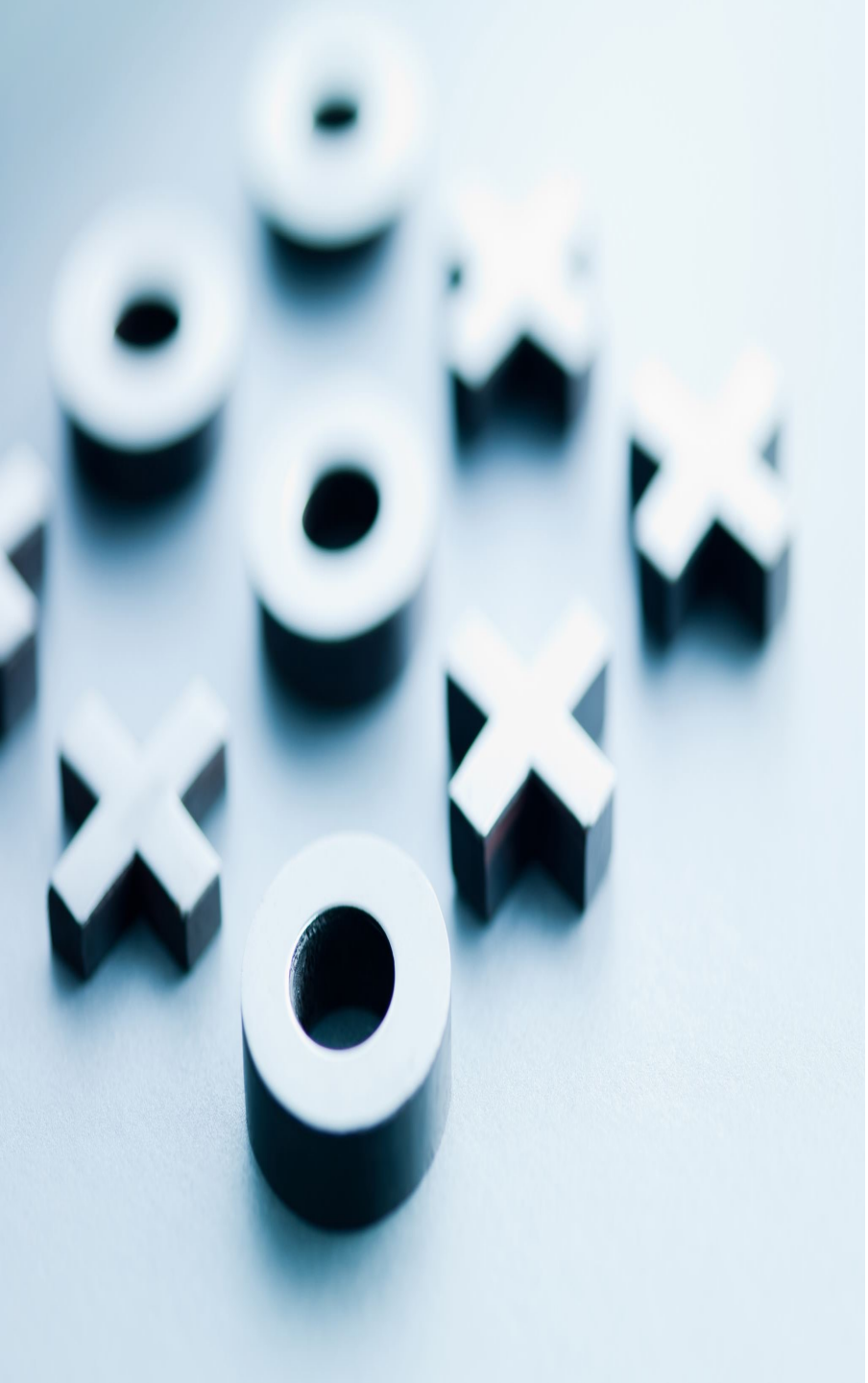
We treat this estimate as the state's value, and the whole table is the learned value function.

State A has higher value than state B, or is considered "better" than state B, if the current estimate of the probability of our winning from A is higher than it is from B.



Assuming we always play Xs, then for all states with three Xs in a row the probability of winning is 1, because we have already won. Similarly, for all states with three Os in a row, or that are filled up, the correct probability is 0, as we cannot win from them.

We set the initial values of all the other states to 0.5, representing a guess that we have a 50% chance of winning.



We then play many games against the opponent.

To select our moves we examine the states that would result from each of our possible moves (one for each blank space on the board) and look up their current values in the table.

Most of the time we move greedily, selecting the move that leads to the state with greatest value, that is, with the highest estimated probability of winning.

Occasionally, however, we select randomly from among the other moves instead.

These are called exploratory moves because they cause us to experience states that we might otherwise never see.

A sequence of moves made and considered during a game can be diagrammed as in Figure 1.

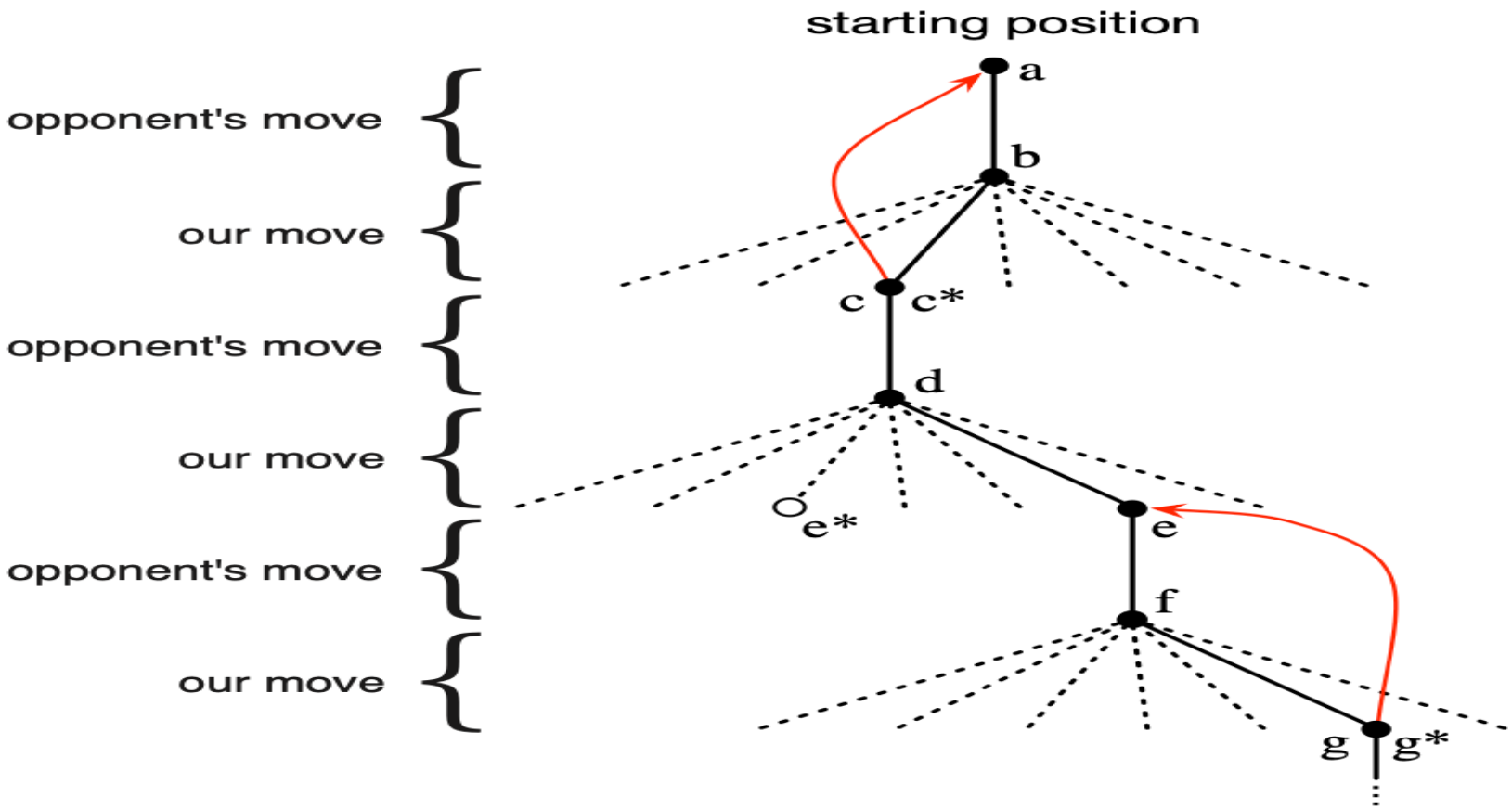


Figure 1: A sequence of tic-tac-toe moves. The solid black lines represent the moves taken during a game; the dashed lines represent moves that we (our reinforcement learning player) considered but did not make. Our second move was an exploratory move, meaning that it was taken even though another sibling move, the one leading to e *, was ranked higher. Exploratory moves do not result in any learning, but each of our other moves does, causing updates as suggested by the red arrows in which estimated values are moved up the tree from later nodes to earlier nodes as detailed in the text.

While we are playing, we change the values of the states in which we find ourselves during the game.

We attempt to make them more accurate estimates of the probabilities of winning. To do this, we “back up” the value of the state after each greedy move to the state before the move, as suggested by the arrows in Figure 1.

More precisely, the current value of the earlier state is updated to be closer to the value of the later state.

This can be done by moving the earlier state's value a fraction of the way toward the value of the later state.

If we let St denote the state before the greedy move, and St+1 the state after the move, then the update to the estimated value of St, denoted V (St), can be written as

$$V(S_t) \leftarrow V(S_t) + \alpha\Big[V(S_{t+1}) - V(S_t)\Big],$$

Where α is small positive fraction called the step-size parameter, which influences the rate of learning.

This update rule is an example of a temporal-difference learning method, so called because its changes are based on a difference, V (S_{t+1})V (S_t), between estimates at two successive times.

The method described performs quite well on this task.

For example, if the step-size parameter is reduced properly over time, then this method converges, for any fixed opponent, to the true probabilities of winning from each state given optimal play by our player.

Furthermore, the moves then taken (except on exploratory moves) are in fact the optimal moves against this (imperfect) opponent.

In other words, the method converges to an optimal policy for playing the game against this opponent.

If the step-size parameter is not reduced all the way to zero over time, then this player also plays well against opponents that slowly change their way of playing.

This example illustrates the differences between evolutionary methods and methods that learn value functions.

To evaluate a policy an evolutionary method holds the policy fixed and plays many games against the opponent, or simulates many games using a model of the opponent.

The frequency of wins gives an unbiased estimate of the probability of winning with that policy, and can be used to direct the next policy selection.

But each policy change is made only after many games, and only the final outcome of each game is used: what happens during the games is ignored.

For example, if the player wins, then all of its behavior in the game is given credit, independently of how specific moves might have been critical to the win.

Credit is even given to moves that never occurred! Value function methods, in contrast, allow individual states to be evaluated.

In the end, evolutionary and value function methods both search the space of policies, but learning a value function takes advantage of information available during the course of play.

This simple example illustrates some of the key features of reinforcement learning methods.

First, there is the emphasis on learning while interacting with an environment, in this case with an opponent player.

Second, there is a clear goal, and correct behavior requires planning or foresight that takes into account delayed effects of one's choices.

For example, the simple reinforcement learning player would learn to set up multi-move traps for a shortsighted opponent.

It is a striking feature of the reinforcement learning solution that it can achieve the effects of planning and lookahead without using a model of the opponent and without conducting an explicit search over possible sequences of future states and actions.

While this example illustrates some of the key features of reinforcement learning, it is so simple that it might give the impression that reinforcement learning is more limited than it really is.

Although tic-tac-toe is a two-person game, reinforcement learning also applies in the case in which there is no external adversary, that is, in the case of a "game against nature."

Exercises



- **Self-Play Suppose,**

instead of playing against a random opponent, the reinforcement learning algorithm described played against itself, with both sides learning. What do you think would happen in this case? Would it learn a different policy for selecting moves?

- **Symmetries**

Many tic-tac-toe positions appear different but are really the same because of symmetries. How might we amend the learning process described to take advantage of this? In what ways would this change improve the learning process? Now think again. Suppose the opponent did not take advantage of symmetries. In that case, should we? Is it true, then, that symmetrically equivalent positions should necessarily have the same value?

- **Greedy Play**

Suppose the reinforcement learning player was greedy, that is, it always played the move that brought it to the position that it rated the best. Might it learn to play better, or worse, than a nongreedy player? What problems might occur?

- **Learning from Exploration**

Suppose learning updates occurred after all moves, including exploratory moves. If the step-size parameter is appropriately reduced over time (but not the tendency to explore), then the state values would converge to a different set of probabilities. What (conceptually) are the two sets of probabilities computed when we do, and when we do not, learn from exploratory moves? Assuming that we do continue to make exploratory moves, which set of probabilities might be better to learn? Which would result in more wins?

- **Other Improvements**

Can you think of other ways to improve the reinforcement learning player? Can you think of any better way to solve the tic-tac-toe problem as posed?

H.W

- **There are four important characteristics in reinforcement learning. What are they? What is the characteristic of each one of them?**
- **Describe the tic tac toe game, how can implement by reinforcement learning? explain the factors of reinforcement learning in the game, what's mean each factor in this game?**

References

Sutton, Richard S., and Andrew G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.